

מבנה נתונים תור - חלק א'

היכרות עם מבנה נתונים תור

פעולות על תור

נקודות דמיון ושוני בין תור לרשימה

תור בפייתון (שימוש בספרייה מובנית queue)



נפגשנו פעם

למי מאיתנו לא יצא פעם לעמוד בתור? בין אם בתור לקופה בסופרמרקט, תור לרכבת הרים בלונה-פארק או...תור לסרט בסינמה סיטי? כולנו עשינו את זה! בחיים האמיתיים לא כל כך כיף לחכות הרבה זמן בתור, אבל אין ספק שזה עושה הרבה סדר ומונע בלאגן - ולא רק בחיים האמיתיים, אלא גם בתכנות. אבל על זה נדבר בהמשך.

תור עובד בצורה פשוטה מאוד: יש שורה של אנשים המחכים לקבל דבר-מה, כגון שירות או מזון, והם עומדים לפי סדר הקבלה המיועד. בתמונה שמופיעה למעלה, האיש המבוגר הוא הראשון לקבל שירות, לאחריו האישה שמחזיקה את היד של הבת שלה וכך הלאה. לאחר שמי שבראש התור מסיים לקבל שירות, הוא "יוצא מהתור" והתור מתקדם. אם מישהו רוצה להצטרף לתור, עליו להמתין בסוף התור. ובקיצור: מי שנכנס ראשון, יוצא ראשון.

אוקיי, הבנו. ואיך כל זה קשור לתכנות?

בתכנות, קיים מבנה נתונים בשם "תור" (באנגלית: Queue) שמתנהג בצורה דומה לתור שאנו מכירים בחיים האמיתיים.

מבנה נתונים זה מזכיר רשימה (list) מאחר שאפשר לשמור בו כמות גדולה של נתונים בזה אחר זה, ויש לו פעולות שהן ייחודיות רק עבורו. להלן רשימת הפעולות:

יצירת-תור-ריק()
הכנסה-לתור()
הוצאה-מהתור()
בדיקה-האם-התור-ריק?()
בדיקת-ערך-בראש-התור()

איך משתמשים בהן ומה הן עושות? אל דאגה, זה ממש פשוט. מיד נדגים שימוש בפעולות בזה אחר זה.

הדגמת שימוש בפעולות של תור

קודם כל, ניצור...תור ריק, כמובן. נניח שאנו רוצים לפתוח תור לדוכן פופקורן בסינמה סיטי. לצורך כך, נקרא לפעולה יצירת-תור-ריק().



מצוין, יצרנו תור ריק לדוכן שלנו. עכשיו מגיע הבחור הראשון שרוצה להיכנס לתור - איזה מרגש! זהו בחור מבוגר העונה לשם "אברהם". לצורך כך, נקרא לפעולה הכנסה-לתור("אברהם").



בקלות אברהם שלנו התווסף לתור. כעת, בחורה העונה לשם "נטלי" רוצה להצטרף לתור וגם הבת הקטנה שלה "ליאן". באופן דומה, נקרא לפעולה **הכנסה-לתור** ("נטלי") ולאחר מכן שוב לפעולה **הכנסה-לתור** ("ליאן").



אנו יכולים לראות שפעולת **הכנסה-לתור** תמיד מכניסה מהסוף של התור.

שאלה באמצע הדרך ?

מה לדעתכם יקרה כשנשתמש בפעולה **הוצאה-מהתור** () על התור שלנו?



1. ליאן תצא מהתור
2. נטלי תצא מהתור
3. אברהם יצא מהתור
4. כלום לא יקרה כי לא אמרנו לפעולה את מי להוציא מהתור.

התשובה הנכונה היא 3 - אברהם יצא מהתור. ✓

פעולת הוצאה-מהתור מוציאה נתונים תמיד מראש התור ולכן אברהם הוא זה שיצא ראשון. שימו לב שאי אפשר לקבוע לפעולה את מי להוציא מהתור - היא אוטומטית מוציאה את מי שנמצא כרגע בראש התור.

נחזור לתור שלנו. אחרי שהוצאנו ממנו את אברהם הוא נראה כך:



נוכל להשתמש בפעולה **בדיקה-האם-התור-ריק?** () כדי לבדוק אם יש אנשים בתור. כשנקרא לפעולה **בדיקה-האם-התור-ריק?** () נקבל את התשובה **שקר**, כי הרי התור לא ריק - נטלי וליאן עדיין מחכות לפופקורן שלהן...
נותרה רק פעולה אחת שלא השתמשנו בה והיא **בדיקת-ערך-בראש-התור** (). פעולה זו למעשה שואלת "מי נמצא בראש התור?" מבלי להוציא את הבן אדם מהתור. במקרה שלנו, אם נפעיל את הפעולה, נקבל את התשובה **נטלי**. נטלי עדיין תחכה בתור, כי הפעולה לא מוציאה אותנו ממנו.

סיימנו לעבור על כל הפעולות. איזה כיף!





הרחבה: נקודות דמיון ושוני בין תור לרשימה

הזכרנו קודם לכן שתור מזכיר את מבנה הנתונים רשימה, וכעת הגיע הזמן להבין מה דומה ומה שונה ביניהם.

במה דומים?

אוסף של נתונים - גם תור וגם רשימה הם מבני נתונים שיכולים להכיל כמות גדולה של נתונים. זאת בניגוד למשתנים פשוטים כמו מחרוזת או מספר שלם - שיכולים להכיל נתון אחד בלבד בכל רגע נתון.

פעולות של מבנה הנתונים - גם תור וגם רשימה הם מבני נתונים שאפשר לבצע עליהם פעולות. לדוגמה, על רשימה אפשר להשתמש בפעולה "שרשור" (append) ועל תור אפשר להשתמש בפעולה "הכנס-לתור".

במה שונים?

גישה לנתונים - כפי שאנו יודעים, אפשר לגשת לכל איבר ברשימה באמצעות מיקום. לדוגמה:

```
>>> my_lst[5]
```

בשורת הקוד לעיל אנו ניגשים לאיבר במיקום 6 ברשימה ומדפיסים אותו בלי בעיה. עם זאת, מבנה נתונים תור לא מאפשרת לנו את זה. בתור אפשר לגשת אך ורק לנתון שנמצא בראשו באמצעות הפעולה "בדיקת-ערך-בראש-התור()" או להוציא אותו באמצעות הפעולה "הוצא-מהתור".

שינוי נתונים - אנו יכולים לשנות איברים קיימים ברשימה, לדוגמה:

```
>>> my_lst[5] = "Avraham"
```

בשורת הקוד לעיל אנו ניגשים לאיבר במיקום 6 ברשימה ומשנים את ערכו לערך המחרוזת "אברהם". עם זאת, בתור אין לנו אפשרות לשנות נתונים בתוכו כרצוננו - מה שנכנס לתור כבר בפנים ואין לנו אפשרות לגשת אליו ושנות אותו. בנוסף, איננו יכולים להכניס לתור נתונים איפה שאנחנו רק רוצים - אלא אך ורק מהסוף שלו. בדיוק כמו שאזרחים טובים עומדים בתור מהסוף שלו, ולא עוקפים אף אחד...כולנו יודעים שזה פשוט לא מנומס ;-)

איך כל זה עובד בפיתון?

הכרנו את הפעולות של התור והדגמנו כיצד הן פועלות. כעת הגיע הזמן להבין איך כל זה עובד בפיתון. והאמת היא...שזה כמעט אותו הדבר.

בחוברת זו, נלמד להשתמש בספרייה מובנית בפיתון בשם `queue` שקיימות בה כל הפעולות של תור שראינו קודם לכן.

לפניכם טבלה המציגה את הפעולות שראינו קודם לכן - והמקבילה שלהן בפיתון.

פעולה	איך כותבים אותה בפיתון
יצירת-תור-ריק()	<code>q = queue.Queue()</code>
הכנסה-לתור()	<code>q.put(ערך_שנרצה_להכניס_לתור)</code>
הוצאה-מהתור()	<code>q.get()</code>
בדיקה-האם-התור-ריק?()	<code>q.empty()</code>
מספר האיברים בתור	<code>q.qsize()</code>
הדפסת כל איברי התור	<code>print(q.queue)</code>
בדיקת ערך בראש התור	לא ממומש בספרייה <code>Queue</code>

שימו לב: כדי שתוכלו להשתמש בתור ופעולותיו בתוכנית שלכם, עליכם לייבא את הספרייה queue. עשו זאת באמצעות הפקודה הבאה:

```
import queue
```

קדימה - פתחו פייתון ונסו ליצור תור משלכם. שחקו קצת עם הפעולות והרגישו כיצד הן עובדות! זה קל וכיף.

בתור התחלה, נסו להריץ את התוכנית הפשוטה הבאה.

```
import queue

q = queue.Queue()

q.put(1)
q.put(2)

print(q.get())
```

מה קיבלתם?

התוכנית יוצרת תור ריק בשם q. לאחר מכן היא מכניסה אליו את הערך 1 ואז את הערך 2. בסיום היא מוציאה את הערך בראש התור ומדפיסה אותו - הלא הוא הערך 1.

איך ממשיכים מכאן?

אם סיימתם ואתם מרגישים שאתם שולטים היטב בחומר - עברו לחוברת התרגילים של מבנה נתונים תור חלק א'. בהצלחה!

